(Special Session)

Closing the Loop in LLM-Based Hardware Generation: An Autonomous Agentic Workflow for Robust TPU Design

Deepak Vungarala†, Kartik Pandit†, Gamana Aragonda†, Jeremy McLynch†, Adeola Adeoye-Davids†, Bryan Galecio†, NhatHai Phan†, Abdallah Khreishah†, Ramtin Zand‡, Arnob Ghosh†, Shaahin Angizi†
†New Jersey Institute of Technology, Newark, NJ, USA ‡University of South Carolina, Columbia, SC, USA
E-mails: {dv336,arnob.ghosh,shaahin.angizi}@njit.edu, ramtin@cse.sc.edu

2026 IEEE 44th VLSI Test Symposium (VTS) | 979-8-3315-6337-0/26/$31.00 ©2026 IEEE | DOI: 10.1109/VTS69484.2026.11563407

***Abstract*—Large language models (LLMs) can generate hardware description language (HDL) code from high-level specifications, but most existing approaches remain open loop and require manual recovery when designs fail in simulation, verification, or implementation. This limitation is especially severe for approximate arithmetic, where circuits must satisfy both numerical error bounds and architecture-level constraints. We present ARCADE, a separation-enforced multi-agent framework for architecture-aware generation of approximate arithmetic units. ARCADE combines an independent Design Agent and Verifier Agent, a fixed golden-reference NMED harness for functional acceptance, retrieval-augmented error memory (RAEM) for closed-loop repair, and automated integration into an RTL-to-GDSII flow. Evaluated on 13 approximate-adder specifications within the APTPU datapath, ARCADE improves first-attempt success from 30.8% to 76.9%, solves all specifications within three attempts, and reduces average NMED from 25.6% to 0.7%. The framework and code will be released as open source at https://github.com/ACADLab/ARCADE**

I. Introduction

The rapid growth of Deep Neural Network (DNN) workloads has driven the adoption of domain-specific accelerators that specialize in datapaths for dense linear algebra operations. Architectures such as Tensor Processing Units (TPUs) and systolic-array accelerators achieve high throughput and energy efficiency by exploiting structured dataflow and massive parallelism [1]. Designing these accelerators, however, requires exploring a large hardware design space while satisfying strict power, performance, and area (PPA) constraints.

Approximate computing is one approach to improving accelerator efficiency by relaxing exact arithmetic in error-tolerant workloads. Inference tasks often tolerate bounded numerical deviations, enabling arithmetic units to trade accuracy for reductions in area, power, and latency [2]. Approximate adders are particularly important in this context because they lie on the critical path of multiply–accumulate (MAC) datapaths. Small structural changes in the adder logic, such as truncation, speculative carry propagation, or error compensation, can significantly affect both numerical error and hardware cost.

Recent advances in large language models (LLMs) have demonstrated the ability to synthesize hardware description language (HDL) code directly from natural-language specifications [3]–[8]. While promising, most existing LLM-based hardware-generation approaches operate in an open-loop configuration. Generated RTL is typically validated only after compilation or simulation, and failures must be corrected manually by human designers. However, the works of no-human-in-loop (NHIL) promise and demonstrate the ability of LLM to design with feedback without manual intervention. They are designed as autonomous systems and do not integrate with the architectural topology [9]–[11]. This limitation becomes more severe for approximate arithmetic design, where the objective is not only functional correctness but also compliance with quantitative error bounds and compatibility with downstream implementation flows. These challenges highlight a key gap in current LLM-driven electronic design automation (EDA): existing pipelines lack mechanisms to enforce numerical correctness, automatically recover from generation failures, and ensure that synthesized circuits integrate into realistic accelerator architectures. A practical system must therefore combine code generation with verification against fixed mathematical metrics, structured repair mechanisms, and architecture-aware implementation evaluation.

To address this problem, we introduce **ARCADE (Architecture aware Retrieval and Closed loop Approximate Design Engine)**, a multi-agent framework for autonomous generation and verification of approximate arithmetic units. ARCADE operates from sparse specifications containing only `bit_width`, `nmed_target`, and `area_priority`. A Design Agent synthesizes candidate RTL implementations, while an independent Verifier Agent generates a black-box testbench from the same specification without access to the generated design. Functional correctness is determined by a fixed golden-reference harness that computes normalized mean error distance (NMED) against exact arithmetic. When failures occur, ARCADE performs closed-loop repair guided by retrieval-augmented error memory (RAEM), which stores prior error–fix–outcome patterns and provides them as context for subsequent design iterations. Fig. 1 illustrates the ARCADE workflow. The main contributions of this work are: (i) the ARCADE framework, a separation-enforced multi-agent pipeline for architecture-aware generation of approximate arithmetic units; (ii) a verification methodology combining independent testbench generation with a fixed golden-reference NMED evaluator; (iii) a retrieval-augmented error memory (RAEM) that enables persistent learning across design attempts; and (iv) an end-to-end evaluation pipeline that integrates generated

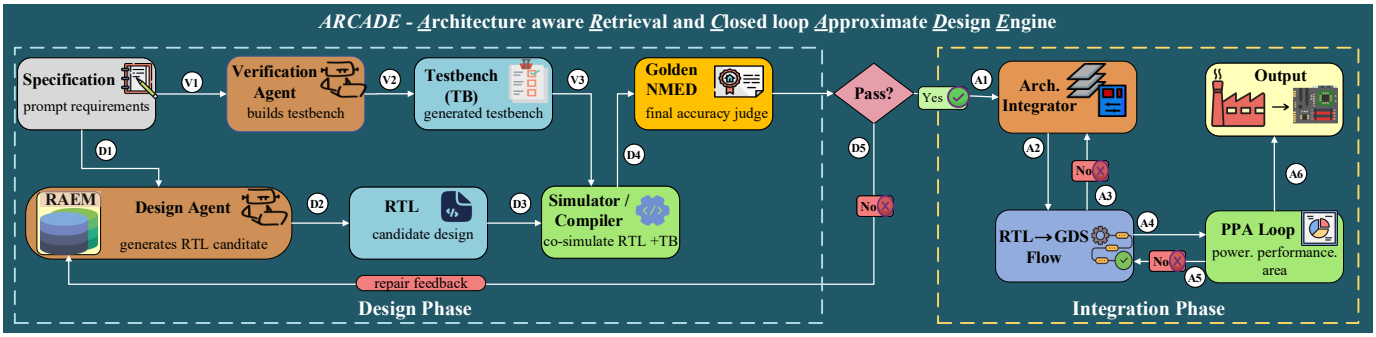

Fig. 1. ARCADE framework.

circuits into a realistic RTL-to-GDSII accelerator implementation flow.

II. Background

LLMs for Hardware Design. Recent work has explored the use of LLM's to automate multiple stages of the hardware design workflow. Early efforts demonstrate that LLMs can generate HDL and high-level synthesis (HLS) implementations from textual specifications. Frameworks such as VeriGen [3] and ChatEDA [4] investigate LLM-assisted RTL generation within the broader RTL-to-GDSII pipeline, while systems such as AssertLLM [12] and UVLLM [13] focus on verification tasks by generating SystemVerilog assertions and assisting automated RTL debugging and repair. A second line of work studies LLM-driven hardware generation and optimization using simulation feedback. Approaches including ChipGPT [14] and AutoChip [15] iteratively refine generated Verilog designs through simulation-guided feedback loops. Complementary efforts investigate data and framework infrastructure for hardware generation. For example, TPU-Gen [16] and MG-Verilog [17] introduce datasets designed to support large-scale HDL generation, while Chip-Chat [6], MEV-LLM [18], and DeepCircuitX [19] explore interactive design environments, multi-expert architectures, and Chain-of-Thought reasoning for RTL synthesis and early PPA estimation. Additional systems such as RTLLM [20], GPT4AIGChip [9], and LLMCompass [21] demonstrate the potential of LLMs to accelerate architectural exploration and improve hardware design productivity. Despite this progress, several challenges remain. LLM-based EDA pipelines still depend heavily on prompt engineering, domain-specific datasets, and model adaptation, while hallucinations and correctness issues continue to affect generated designs [4]. As a result, many approaches rely on in-context learning (ICL) to improve generation quality [22]. Parallel efforts in analog design have begun exploring similar ideas. Systems such as AnalogCoder [23], SPICEPilot [24], and Masala-CHAI [25] demonstrate early attempts to generate SPICE netlists using prompt engineering and ICL. Emerging directions further investigate LLM-assisted hardware paradigms, including in-memory computing architectures such as LIMCA [11].

Approximate Adders and NMED. Approximate arithmetic reduces area and power by relaxing exact computation, making it attractive for error tolerant inference workloads [26]–[28]. Approximate adders span several recurring design patterns, including lower bit OR substitution, carry speculation, truncation, and explicit error compensation. These structures define a broad trade-off surface between hardware cost and numerical fidelity. Evaluation in this setting requires a metric tied to exact arithmetic rather than to a chosen approximate baseline. The framework, therefore, uses NMED as the sole functional acceptance criterion prior to physical integration. Let P_{exact} denote the exact sum and P_{approx} the approximate output. Over N test vectors, the average mean error distance is

$$\text{MED}_{\text{avg}} = \frac{1}{N}\sum_{i=1}^{N} |P_{\text{exact},i} - P_{\text{approx},i}|. \tag{1}$$

NMED is then defined as

$$\text{NMED} = \frac{\text{MED}_{\text{avg}}}{|\text{Avg}(P_{\text{exact}})|}. \tag{2}$$

The key property is that the reference is an exact addition implemented in a fixed C++ harness, not an LLM-generated model, nor an open-sourced or human-written approximate circuit. A generated adder therefore passes only when its measured error remains below the target bound. This makes NMED an appropriate acceptance metric for a closed-loop pipeline whose goal is to synthesize approximate hardware from sparse constraints and then integrate only numerically validated designs into the APTPU [29] flow. Fig. 1 summarizes the end-to-end workflow from sparse specification to golden-harness validation and downstream PPA evaluation.

III. ARCADE: Architecture aware Retrieval and Closed Loop Approximate Design Engine

ARCADE enables automated synthesis of approximate arithmetic circuits by combining LLM generation, independent verification, retrieval-augmented repair, and architecture-level implementation evaluation within a unified closed-loop workflow. The framework is designed to autonomously explore approximation strategies, produce synthesizable hardware implementations, and iteratively repair candidate circuits that violate numerical accuracy or implementation constraints.

Fig. 1 illustrates the ARCADE pipeline. The framework comprises two stages, separated by a final verification gate. The *Design Phase* generates candidate approximate circuits and verifies their numerical correctness using a fixed golden-reference metric. The *Integration Phase* inserts verified circuits into the accelerator architecture and evaluates them through a full RTL-to-GDSII physical-design flow. This separation is intentional: designs that violate numerical constraints are never implemented, whereas designs that are numerically correct but

physically infeasible are rejected during architectural evaluation.

Parallel Design and Verification. The ARCADE pipeline begins from a minimal specification containing three fields: `bit_width`, `nmed_target`, and `area_priority`. From this specification, two independent branches are launched simultaneously, initiating both verification and design flows. The verification path begins with the Verification Agent (V1), which receives the specification and generates a self-checking testbench (V2). The generated testbench treats the approximate circuit as a black box, producing input vectors and computing error measurements without observing any candidate RTL (from (D3)) (V3) forwarded to the simulator. The resulting testbench is then forwarded to the golden numerical evaluator (V3), which serves as the authoritative judge of functional correctness.

Retrieval-Augmented Design Generation. In parallel with the verification branch, the specification is routed to the design branch through RAEM (D1). RAEM maintains an append-only database of prior repair outcomes that stores error signatures, design context, and observed results. When a new design attempt begins, RAEM retrieves historically similar failure cases and injects them into the prompt context of the Design Agent. The Design Agent (D2) then synthesizes a candidate Verilog module implementing an approximate circuit. The framework deliberately avoids providing circuit-family hints or reference topologies, forcing the LLM to autonomously select an approximation strategy. Possible strategies include lower bit truncation, OR-gating of carry chains, speculative carry propagation, or hybrid approximate structures. The generated module forms the candidate RTL design (D3), which is subsequently compiled and co-simulated with the verifier-generated testbench using a simulator or compiler environment (D4).

Golden NMED Verification. The numerical correctness of the candidate design is determined by a fixed golden-reference harness that evaluates the normalized mean error distance (NMED). Let P_{exact} denote the exact arithmetic result and P_{approx} the output produced by the candidate design. At the decision node (the final verification gate), the candidate design is accepted if $\text{NMED} \leq$ `nmed_target` (A1). Otherwise, the verification harness generates structured repair feedback that is logged into RAEM and routed back to the Design Agent, triggering a new generation attempt informed by previous failures (D5). This closed-loop mechanism allows ARCADE to progressively refine candidate circuits until the numerical constraint is satisfied or the iteration limit is reached.

Architecture Integration and Implementation Evaluation. Once a candidate circuit satisfies the golden NMED constraint, it enters the Integration Phase. The Architecture Integrator (A1) inserts the generated RTL module into the accelerator datapath. In the current implementation, this integration targets the APTPU architecture [29]; furthermore, the framework is designed to support other accelerator designs. Following integration, the design is passed to the RTL-to-GDS flow (A2), which executes synthesis, placement, routing, and timing closure through an automated physical-design toolchain. If the design is not integrated, feedback is sent to Architecture Integrator (A3) to retry with the error log in place. The resulting hardware implementation is evaluated through the architecture-level PPA loop (A4), which measures power, performance, and area characteristics. If the design violates these constraints (A5), the implementation is rejected, and the feedback re-enters the RTL-to-GDS to guide subsequent design attempts, such as timing closure and power gating. Only the designs that satisfy both the NMED accuracy constraint and the architectural PPA requirements are generated as the final hardware output (A6).

Closed-Loop Architecture-Aware Hardware Generation. By combining prompt-level separation between design and verification agents, a fixed numerical oracle for functional acceptance, a retrieval-guided repair memory, and a dual-stage verification-and-implementation evaluation pipeline, ARCADE forms an autonomous system for approximate hardware generation. Unlike conventional approximate circuit design workflows that rely on manually curated circuit families or parameter sweeps, ARCADE explores the approximation design space directly from sparse specifications while ensuring that generated circuits satisfy both computational accuracy and architectural implementation constraints.

IV. Experimental Evaluation

Benchmark and Metrics. The benchmark comprises 13 approximate-adder specifications (A1–A13), each defined by three fields: `bit_width` $\in \{8, 16, 32\}$, `nmed_target` $\in [0.005, 0.10]$, and `area_priority` $\in \{$high, medium, low$\}$. No circuit-family hint, topology label, or reference implementation is included; the LLM must determine an approximation strategy solely from the numerical constraint and the qualitative area preference. This deliberate omission prevents the model from reconstructing known approximate-adder families and instead evaluates its capacity for constraint-driven circuit invention. Generated designs are assessed along four axes: *Pass@*k, the fraction of specifications solved within k generation attempts; *average iterations*, the mean number of repair steps required to reach a valid design; *average NMED*, the mean normalized mean error distance across accepted designs as defined in Eq. (2); and *GDS success*, the percentage of accepted designs that complete the full RTL-to-GDS implementation flow without violating PPA constraints. To isolate the contribution of each framework component, four cumulative configurations are evaluated. *ARCADE-Vanilla* performs single-shot generation with no repair loop and no retrieval memory, establishing the baseline capability of the underlying model. *ARCADE-Repair* adds the closed-loop repair mechanism, feeding structured NMED failure diagnostics back to the Design Agent across up to five iterations. *ARCADE-Memory* augments repair with RAEM retrieval, prepending the top-k historically similar failure contexts to each repair prompt. *ARCADE-Full* activates the complete pipeline, including sequential learning across specification runs and architecture integration via the OpenROAD [30] flow.

TABLE I
FRAMEWORK PROGRESSION ACROSS ARCADE CONFIGURATIONS.

Configuration	Pass@1	Pass@3	Pass@5	Avg. Iter.	Avg. NMED	GDS
ARCADE-Vanilla	30.8%	30.8%	30.8%	1.00	25.57%	12.4%
ARCADE-Repair	30.8%	61.5%	69.2%	2.85	6.56%	69.2%
ARCADE-Memory	53.8%	53.8%	61.5%	2.77	8.32%	61.5%
ARCADE-Full	**76.9%**	**100%**	**100%**	**1.31**	**0.70%**	**100%**

Table I reports results across all four configurations on the 13-specification benchmark. Single-shot generation alone (*ARCADE-Vanilla*) succeeds on 30.8% of specifications, and the designs it produces have a mean NMED of 25.57%, nearly 36× the median target in the benchmark, indicating that the model can occasionally produce structurally valid approximate adders but lacks the self-correction mechanism to satisfy tight error budgets. The GDS success rate of 12.4% further confirms that first-attempt designs are rarely suitable for physical integration. Introducing iterative repair (*ARCADE-Repair*) raises Pass@5 to 69.2% and reduces mean NMED by more than 3.9× relative to *ARCADE-Vanilla*. The GDS success rate increases from 12.4% to 69.2%, demonstrating that repair feedback produces designs that are not merely numerically compliant but also physically realizable. *ARCADE-Memory* improves Pass@1 to 53.8%, confirming that historical repair context reduces the number of failed first attempts, though its Pass@5 of 61.5% is lower than *ARCADE-Repair* in isolation, a result examined further in Section IV-B. *ARCADE-Full* achieves the strongest results across all metrics: Pass@1 of 76.9%, Pass@3 and Pass@5 both at 100%, mean NMED of 0.70%, average iteration count of 1.31, and 100% GDS success. The reduction in average iterations from 2.85 (*ARCADE-Repair*) to 1.31 (*ARCADE-Full*) indicates that RAEM-guided retrieval, when combined with sequential cross-specification learning, substantially concentrates successful designs toward earlier attempts.

A. Designer-Agent Ablation

To isolate the effect of the generative model, the verifier is fixed to DeepSeek-V3.2 Chat [31] and the designer is varied between DeepSeek-Reasoner [32] and DeepSeek-V3.2 Chat. Table II reports results under both the single-shot baseline and the full ARCADE pipeline. DeepSeek-Reasoner achieves the higher Vanilla Pass@1 (30.8% vs. 15.4%), indicating stronger zero-shot circuit synthesis. This advantage is consistent with its explicit chain-of-thought reasoning mechanism, which allows more deliberate strategy selection before committing to a topology. Under *ARCADE-Full*, however, DeepSeek-V3.2 Chat surpasses it on every metric: Pass@1 rises to 53.8% against 46.2%, Pass@5 reaches 92.3% against 84.6%, and the mean iteration count falls to 2.15 from 2.46. The inversion suggests that some models derive greater benefit from iterative repair and retrieval guidance than from stronger initial reasoning: the ability to effectively incorporate structured feedback and historical context matters more than single-shot generation quality once the full loop is engaged.

B. Verifier-Agent Ablation

With the designer fixed to DeepSeek-V3.2 Chat, the verifier is varied across five models to assess how the quality of generated testbenches and diagnostic feedback affects downstream design success. Table III reports Repair Pass@5, Memory Pass@5, Full Pass@5, and *average NMED* for each verifier. Model abbreviations used throughout the table are defined in the caption. DeepSeek-V3.2 Chat as verifier achieves the strongest Full Pass@5 (92.3%) and the lowest *average NMED* (0.73%), suggesting that consistent model pairing, using the same family for both agent roles, produces the most compatible feedback signals. Claude Sonnet 4.5 [33] and DeepSeek-Reasoner achieve the highest Repair Pass@5 of 84.6%, indicating that reasoning-capable models generate more actionable failure diagnostics during repair iterations. However, both exhibit elevated *average NMED* values (2.34% and 8.16%, respectively) under the full pipeline, implying that their testbench strategies, while effective at catching compilation errors, are less precise at guiding the Design Agent toward tight numerical targets. DeepSeek-R1 Free [34] and Qwen3-Coder 480B [35] achieve comparable Pass@5 rates in the Memory and Full conf. but with higher mean NMED, consistent with less discriminative verification feedback.

TABLE II
DESIGNER-AGENT ABLATION WITH VERIFIER FIXED TO DEEPSEEK-V3.2 CHAT (DSC) [31]. DSR = DEEPSEEK-REASONER [32].

Designer	Vanilla Pass@1	Full Pass@1	Full Pass@5	Avg. Iter.
DSR (Reasoner)	**30.8%**	46.2%	84.6%	2.46
DSC (V3.2 Chat)	15.4%	**53.8%**	**92.3%**	**2.15**

TABLE III
VERIFIER-AGENT ABLATION WITH DESIGNER FIXED TO DEEPSEEK-V3.2 CHAT [31]. CS = CLAUDE SONNET 4.5 [33]; DSR-F = DEEPSEEK-R1 FREE [34]; QC = QWEN3-CODER 480B [35].

Verifier	Repair P@5	Memory P@5	Full P@5	Avg. NMED
DSC	76.9%	84.6%	**92.3%**	**0.73%**
CS	**84.6%**	76.9%	84.6%	2.34%
DSR	**84.6%**	**84.6%**	76.9%	8.16%
DSR-F	76.9%	**84.6%**	76.9%	6.08%
QC	61.5%	**84.6%**	84.6%	2.71%

V. CONCLUSION

This work introduced ARCADE, a multi-agent framework for architecture-aware generation of approximate arithmetic circuits. The framework combines the separation of generation and verification agents, a golden-reference NMED evaluator for functional acceptance, a retrieval-augmented error memory for closed-loop repair, and architecture-level evaluation via an RTL-to-GDSII flow. Experiments on 13 specifications show that ARCADE significantly improves reliability over single-shot LLM generation, achieving a 76.9% first-attempt success rate and solving all specifications within three attempts while reducing average NMED from 25.6% to 0.7%.

ACKNOWLEDGMENTS

This work is supported in part by Synopsys Research Gift from Academic & Research Alliances (SARA) and Product Management & Markets Group (PMG) and NJIT Grace Hopper Artificial Intelligence Research Institute (GHAIRI) Seed Grant.

REFERENCES

[1] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proceedings of the 44th annual international symposium on computer architecture*, 2017, pp. 1–12.

[2] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," *Proceedings of the IEEE*, vol. 108, no. 12, pp. 2108–2135, 2020.

[3] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 3, pp. 1–31, 2024.

[4] H. Wu *et al.*, "Chateda: A large language model powered autonomous agent for eda," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.

[5] D. L. V. D. Vungarala, "Gen-acceleration: Pioneering work for hardware accelerator generation using large language models," Master's Thesis, New Jersey Institute of Technology, 2023, https://digitalcommons.njit.edu/theses/2376.

[6] J. Blocklove, S. Garg, R. Karri, and H. Pearce, "Chip-chat: Challenges and opportunities in conversational hardware design," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 2023, pp. 1–6.

[7] M. Nazzal, K. Nguyen, D. Vungarala, R. Zand, S. Angizi, H. Phan, and A. Khreishah, "Fedchip: Federated llm for artificial intelligence accelerator chip design," in *2025 IEEE International Conference on LLM-Aided Design (ICLAD)*. IEEE, 2025, pp. 93–99.

[8] D. Vungarala, M. Nazzal, M. Morsali, C. Zhang, A. Ghosh, A. Khreishah, and S. Angizi, "Sa-ds: A dataset for large language model-driven ai accelerator design generation," in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2025, pp. 1–4.

[9] Y. Fu *et al.*, "Gpt4aigchip: Towards next-generation ai accelerator design automation via large language models," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.

[10] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, "Verigen: Large language model assisted verilog generation," *arXiv preprint arXiv:2311.04887*, 2023.

[11] D. Vungarala, M. H. Amin, P. Mercati, A. Ghosh, A. Roohi, R. Zand, and S. Angizi, "Limca: Llm for automating analog in-memory computing architecture design exploration," *arXiv preprint arXiv:2503.13301*, 2025.

[12] W. Fang, M. Li, M. Li, Z. Yan, S. Liu, H. Zhang, and Z. Xie, "Assertllm: Generating and evaluating hardware verification assertions from design specifications via multi-llms," *arXiv:2402.00386v1*, 2024.

[13] Y. Hu, J. Ye, K. Xu, J. Sun, S. Zhang, X. Jiao, D. Pan, J. Zhou, N. Wang, W. Shan *et al.*, "Uvllm: An automated universal rtl verification framework using llms," *arXiv preprint arXiv:2411.16238*, 2024.

[14] K. Chang, Y. Wang, H. Ren, M. Wang, S. Liang, Y. Han, H. Li, and X. Li, "Chipgpt: How far are we from natural language hardware design," *arXiv preprint arXiv:2305.14019*, 2023.

[15] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, "Autochip: Automating hdl generation using llm feedback," *arXiv preprint arXiv:2311.04887*, 2023.

[16] D. Vungarala, M. E. Elbtity, S. Syed, S. Alam, K. Pandit, A. Ghosh, R. Zand, and S. Angizi, "Tpu-gen: Closing the loop in llm-based hardware generation for tensor processing units," 2025. [Online]. Available: https://arxiv.org/abs/2503.05951

[17] Y. Zhang, Z. Yu, Y. Fu, C. Wan, and Y. C. Lin, "MG-Verilog: Multi-grained Dataset Towards Enhanced LLM-assisted Verilog Generation," *2024 IEEE LLM Aided Design Workshop (LAD)*, 2024.

[18] B. Nadimi and H. Zheng, "A multi-expert large language model architecture for verilog code generation," *arXiv preprint arXiv:2404.08029*, 2024.

[19] Z. Li, C. Xu, Z. Shi, Z. Peng, Y. Liu, Y. Zhou, L. Zhou, C. Ma, J. Zhong, X. Wang *et al.*, "Deepcircuitx: A comprehensive repository-level dataset for rtl code understanding, generation, and ppa analysis," *arXiv preprint arXiv:2502.18297*, 2025.

[20] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "Rtllm: An open-source benchmark for design rtl generation with large language model," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 722–727.

[21] H. Zhang, A. Ning, R. B. Prabhakar, and D. Wentzlaff, "LLMCompass: Enabling Efficient Hardware Design for Large Language Model Inference," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*.

[22] D. Dai *et al.*, "Why can gpt learn in-context? language models implicitly perform gradient descent as meta-optimizers," *arXiv preprint arXiv:2212.10559*, 2022.

[23] Y. Lai *et al.*, "Analogcoder: Analog circuit design via training-free code generation," 2024.

[24] D. Vungarala, S. Alam, A. Ghosh, and S. Angizi, "Spicepilot: Navigating spice code generation and simulation with ai guidance," *arXiv preprint arXiv:2410.20553*, 2024.

[25] J. Bhandari *et al.*, "Masala-chai: A large-scale spice netlist dataset for analog circuits by harnessing ai," 2025. [Online]. Available: https://arxiv.org/abs/2411.14299

[26] S. Mittal, "A survey of techniques for approximate computing," *ACM Comput. Surveys (CSUR)*, vol. 48, pp. 1 – 33, 2016.

[27] M. S. Ansari, B. F. Cockburn, and J. Han, "An improved logarithmic multiplier for energy-efficient neural computing," *IEEE Transactions on Computers*, vol. 70, no. 4, pp. 614–625, 2021.

[28] V. Mrazek, R. Hrbacek, Z. Vasicek, and L. Sekanina, "Evoapprox8b: Library of approximate adders and multipliers for circuit design and benchmarking of approximation methods," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*. IEEE, 2017, pp. 258–261.

[29] M. E. Elbtity, P. S. Chandarana, B. Reidy, J. K. Eshraghian, and R. Zand, "Aptpu: Approximate computing based tensor processing unit," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 12, pp. 5135–5146, 2022.

[30] (2018) Openroad. [Online]. Available: https://github.com/The-OpenROAD-Project/OpenROAD

[31] DeepSeek, "Deepseek," 2026, accessed: 2026-03-15. [Online]. Available: https://www.deepseek.com/en/

[32] DeepSeek-AI, D. Guo, D. Yang, H. Zhang *et al.*, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, January 2025, model accessed as `deepseek-reasoner` via DeepSeek API. [Online]. Available: https://arxiv.org/abs/2501.12948

[33] Anthropic, "Claude sonnet 4.5," https://www.anthropic.com/news/claude-sonnet-4-5, September 2025, model string: `claude-sonnet-4-5-20250929`. Accessed: 2026-03-15.

[34] DeepSeek-AI, D. Guo, D. Yang, H. Zhang *et al.*, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *Nature*, September 2025, model accessed as `DeepSeek-R1` via free tier at https://chat.deepseek.com. [Online]. Available: https://arxiv.org/abs/2501.12948

[35] Qwen Team, "Qwen3 technical report," May 2025, model: `Qwen3-Coder-480B-A35B-Instruct`. 480B total parameters, 35B activated (MoE). [Online]. Available: https://arxiv.org/abs/2505.09388